Lecture Slip, CS 235
Nov 5, 2026, Thursday
Shunting-Yard Algorithm

Name:	
EmpID:	

1. **Three notations.** Consider the infix expression `6 + 2 * (5 - 1)`.

   (a) Write it in **postfix** and in **prefix**.

   (b) Evaluate the postfix form with one stack. Write the stack (bottom to top) just before the `+` is processed, and the final value.

   Answer: (a) Postfix: `6 2 5 1 - * +`. Prefix: `+ 6 * 2 - 5 1`. (The tree has `+` at the root, `6` on the left, and `*` on the right with children `2` and `-` (`5`, `1`). Postfix lists left, right, then the operator; prefix lists the operator first.)

   (b) Just before `+`: the stack is `[6, 8]` (the `-` made $5-1=4$, the `*` made $2\times 4=8$). The `+` pops 8 first (right operand), then 6: $6+8=$ **14**.

2. **Operand order.** Evaluate the postfix expression `20 6 2 - /`.

   (a) Show the stack (bottom to top) after each token.

   (b) In `evalPostfix`, `b` is popped first and `a` second. Suppose a student reverses the operands of **both** `-` and `/` (computing `b - a` and `b / a`). What result does the code produce?

   Answer: (a) `20` → `[20]`; `6` → `[20 6]`; `2` → `[20 6 2]`; `-` pops $b=2$, $a=6$, pushes $6-2=4$ → `[20 4]`; `/` pops $b=4$, $a=20$, pushes $20/4=5$ → `[5]`. The value is **5**.

   (b) With the operands swapped, `-` computes $b-a=2-6=-4$, and then `/` computes $b/a=-4/20=$ $-$**0.2** (correct: 5). The first value popped is the **right** operand. It matters for `-` and `/`, not for `+` and `*`.

3. **One character matters.** The operator loop in `toPostfix` uses `prec(ops.top()) >= prec(t)`. A student changes it to `prec(ops.top()) > prec(t)`.

   (a) What postfix does the changed code produce for `16 - 4 - 2`, and what does it evaluate to? What is the correct result?

   (b) Which property of `-` does the `>=` test protect?

   Answer: (a) The changed code leaves the first `-` on the stack when the second `-` arrives, so the output is `16 4 2 - -`, which evaluates to $16-(4-2)=$ **14**. The correct output is `16 4 - 2 -`, which gives $(16-4)-2=$ **10**.

   (b) **Left associativity**: operators of equal precedence are grouped from the left, so the earlier one must be output first.

4. **Shunting-yard trace.** Convert `8 - 3 * 2 + 1` to postfix. Fill in the table. In "Action", record the action or sequence of actions (output, push, pop); an incoming operator may cause several stacked operators to be popped. The last row is the end of the input.

Step	Token	Action	Output	Stack (bottom to top)
1	`8`	output	`8`	
2	`-`	push `-`	`8`	`-`
3	`3`	output	`8 3`	`-`
4	`*`	push `*`	`8 3`	`- *`
5	`2`	output	`8 3 2`	`- *`
6	`+`	pop `*`, pop `-`, push `+`	`8 3 2 * -`	`+`
7	`1`	output	`8 3 2 * - 1`	`+`
8	*end*	pop `+`	`8 3 2 * - 1 +`	

Which row shows the rule "pop operators of **higher or equal** precedence"? What value does the postfix result give?

Answer: The table above is the key. **Row 6**: the new `+` pops `*` (higher precedence) and then `-` (equal precedence, left-associative), and only then is pushed. Result: `8 3 2 * - 1 +`, which evaluates to $8 - 6 + 1 = \mathbf{3}$.

5. **Pace survey.** Please help me set the pace of the course. Circle one.

Too **slow** (I can go faster)	**About right** (normal pace)	Too **fast** (please slow down)

Answer: Tally the three circles; there is no correct answer.

6. **Topics to clarify.** Check **all** topics you would like me to explain again next class.

- ☐ Infix, prefix, and postfix notation
- ☐ Evaluating postfix with a stack
- ☐ Which operand is popped first
- ☐ Tokenizing (multi-digit numbers)
- ☐ Shunting-yard: precedence and ties (`>=`)
- ☐ Shunting-yard: parentheses
- ☐ The code: `tokenize`, `evalPostfix`, `toPostfix`
- ☐ Other: _____

Answer: Tally the checked topics and plan the first ten minutes of the next class around the top two.